

Reproducing CycleNet: Time Series Forecasting through Modeling Periodic Patterns

Ferdinando Giordano

ferdinando.giordano@usi.ch

Gianluca Viviano

gianluca.viviano@usi.ch

Alan Copa

alan.copa@usi.ch

Abstract

We reproduce and analyze CycleNet, a time series forecasting method that explicitly models periodic patterns through learnable recurrent cycles. We first reimplement the core Residual Cycle Forecasting mechanism on synthetic periodic data, showing that CycleNet-Linear matches a Linear baseline when a full cycle is visible, but strongly improves when only a partial cycle is observed. We then evaluate the method on ETTh1 using the authors’ data-loading protocol. CycleNet-Linear consistently improves over a direct Linear baseline across prediction horizons, with larger gains for longer horizons. Additional experiments on cycle length, MLP backbones, and a CycleiTransformer ablation show that explicit cycle modeling is beneficial, although the strongest and most stable results in our reproduction are obtained with the simple CycleNet-Linear model.

1 Introduction

Long-term time series forecasting (LTSF) requires predicting a horizon of length H from a look-back window of length L , often over thousands of steps. Many real-world series carry stable daily, weekly, or seasonal cycles, but deep models typically have to extract this structure implicitly from the input window. CycleNet (Lin et al., 2024) instead models periodicity explicitly: a learnable recurrent cycle table is subtracted from the input, a simple backbone forecasts the residual, and the future cycle is added back. This report tests whether this Residual Cycle Forecasting (RCF) mechanism delivers its claimed benefits under a focused reimplementation.

1.1 Context and Problem Statement

Time series forecasting (TSF) is an essential tool across various domains, including weather forecasting, transportation, and energy management, where it facilitates early warnings and proactive planning. Within this field, accurate predictions over extended periods—referred to as Long-term Time Series Forecasting (LTSF)—are particularly valuable.

The fundamental principle that enables forecasting over these long horizons is the presence of inherent, stable periodicity within the data. Unlike short-term forecasting, which can often rely on recent temporal trends and means, long-term predictions depend heavily on understanding these long-range dependencies.

To address LTSF tasks, current state-of-the-art approaches heavily emphasize their capability to extract features with long-term dependencies. Consequently, the field has seen a proliferation of deep and complex architectures. For instance, models such as Informer, Autoformer, and PatchTST leverage the long-distance modeling capabilities of Transformers, while other methods use large convolutional kernels or segment-wise iterations to capture periodic features from historical long sequences. While effective, the primary purpose of constructing these increasingly complex models is ultimately just to extract periodic features from long-range dependencies.

1.2 The Core Motivation

The reliance on deep and complex architectures in current LTSF research raises a fundamental and pragmatic question: if the primary purpose of constructing these intricate models is solely to better extract periodic features from long-range dependencies, why not directly model the patterns?

Many time series datasets exhibit very clear and stable periodic behaviors, such as the daily or weekly consumption patterns seen in electricity data. Rather than forcing a complex neural network to implicitly learn and extract these cycles from massive amounts of historical data, a globally shared segment can be used to represent the periodic pattern directly. By repeating this segment, the cyclic components of a sequence can be continuously and explicitly represented.

Driven by this insight, the authors pioneer the approach of explicitly modeling these inherent periodic patterns to enhance model performance on LTSF tasks. This shift in perspective moves the focus away from building deeper networks and toward simplifying the problem by isolating the predictable, cyclic nature of the data.

1.3 Forecasting Objective

The goal of time series forecasting is to learn a function that maps past observations to future values:

$$f : x_{t-L+1:t} \in \mathbb{R}^{L \times D} \rightarrow \hat{x}_{t+1:t+H} \in \mathbb{R}^{H \times D}$$

where:

- L is the input sequence length,
- H is the forecasting horizon,
- D is the number of variables (channels),
- $x_{t-L+1:t}$ represents the historical sequence,
- $\hat{x}_{t+1:t+H}$ represents the predicted future sequence.

1.3.1 RCF Alignment and Repetition

The Recurrent Cycle Forecasting (RCF) module uses a learnable recurrent cycle:

$$Q \in \mathbb{R}^{W \times D}$$

where W denotes the cycle length. Since W is usually smaller than the input length L and the prediction horizon H , the cycle is repeatedly shifted and extended to match the required dimensions.

The historical cyclic component is defined as:

$$c_{t-L+1:t} = \left[Q^{(t)}, \dots, Q^{(t)}, Q_{0:L \bmod W}^{(t)} \right]$$

Similarly, the future cyclic component is:

$$c_{t+1:t+H} = \left[Q^{(t+L)}, \dots, Q^{(t+L)}, Q_{0:H \bmod W}^{(t+L)} \right]$$

1.3.2 Instance Normalization

To reduce the effect of distribution shifts, CycleNet applies Reversible Instance Normalization (RevIN) without learnable affine parameters.

The normalization step is:

$$\tilde{x}_{t-L+1:t} = \frac{x_{t-L+1:t} - \mu}{\sqrt{\sigma + \epsilon}}$$

After forecasting, the predictions are transformed back using denormalization:

$$\hat{x}_{t+1:t+H} = \hat{x}_{t+1:t+H} \times \sqrt{\sigma + \epsilon} + \mu$$

where:

- μ is the mean of the input window,
- σ is the standard deviation,
- ϵ is a small constant for numerical stability.

1.3.3 Loss Function

The model is trained using the Mean Squared Error (MSE) loss:

$$\mathcal{L} = \|x_{t+1:t+H} - \hat{x}_{t+1:t+H}\|_2^2$$

This loss measures the difference between the ground-truth future values and the predicted sequence.

1.3.4 Autocorrelation Function

To determine the optimal cycle length W , the authors use the Autocorrelation Function (ACF):

$$ACF(k) = \frac{\sum_{t=1}^{N-k} (x_t - \bar{x})(x_{t+k} - \bar{x})}{\sum_{t=1}^N (x_t - \bar{x})^2}$$

where:

- N is the total number of observations,
- x_t is the value at time step t ,
- k is the lag,
- $\bar{x}$ is the mean of the series.

The optimal cycle length W is selected as the lag k corresponding to the highest peak in the ACF plot.

2 Scope of reproducibility

We restrict our reproduction to the core RCF mechanism rather than the full benchmark suite. We test four claims from Lin et al. (2024):

- **Claim 1.** RCF improves over a matched no-cycle backbone (Linear or MLP).

- **Claim 2.** The benefit of RCF grows with the prediction horizon H .
- **Claim 3.** Performance depends on choosing a cycle length W aligned with the data’s dominant periodicity.
- **Claim 4.** RCF is backbone-agnostic and also helps a Transformer-style architecture.

3 Methodology

We reimplemented RCF in PyTorch from the paper’s description, and used the authors’ repository¹ only as a reference and for the ETTh1 data-loading pipeline (`Dataset_ETT_hour`). The advanced `CycleTransformer` experiment uses the authors’ implementation; the matched no-cycle Transformer baseline is our own ablation. Our code and notebooks are available at <https://github.com/ferdagainagainagain/ATML-cyclenet-reimplementation>.

3.1 Model descriptions

Given an input window $x_{t-L+1:t} \in \mathbb{R}^{L \times D}$, every model predicts $\hat{x}_{t+1:t+H} \in \mathbb{R}^{H \times D}$. RCF maintains a learnable cycle table $Q \in \mathbb{R}^{W \times D}$, repeated and aligned via the time-index-modulo- W phase to produce the past and future cyclic components $c_{t-L+1:t}$ and $c_{t+1:t+H}$. The model then forecasts the residual,

$$\hat{x}_{t+1:t+H} = f_{\theta}(x_{t-L+1:t} - c_{t-L+1:t}) + c_{t+1:t+H},$$

with Q initialized to zero and trained jointly with the backbone f_{θ} . The no-cycle baselines drop the $\pm c$ terms. The Linear and MLP backbones are channel-independent with shared parameters; the MLP is a two-layer ReLU network. For the Transformer-style models we use the inverted-Transformer encoder from `CycleTransformer.py`, with $d_{\text{model}}=64$, 4 heads, one encoder layer and $d_{\text{ff}}=128$. All ETTh1 models use RevIN-style instance normalization. No model is pre-trained.

Table 1: Trainable parameter counts at $L=96$, $H=336$, $D=7$, $W=24$.

Model	Parameters
Linear	32,592
CycleNet-Linear	32,760
MLP ($d=256$)	111,184
CycleNet-MLP	111,352

The cycle module adds only WD parameters (here, 168). Per-sample compute is $\mathcal{O}(LHD)$ for the Linear backbones and is dominated by attention for the Transformer-style models.

3.2 Datasets

Synthetic periodic signal. A univariate series of length $T=2000$ with known cycle $p=24$, generated as $x_t = \sin(2\pi t/p) + 0.0005t + \epsilon_t$, with $\epsilon_t \sim \mathcal{N}(0, 0.15^2)$. Chronological 80/20 train/test split; sliding windows; cycle index = $t \bmod p$.

ETTh1. 17,420 hourly observations and $D=7$ variables (Lin et al., 2024). We use the multivariate setting (`features="M"`). The authors’ loader applies the standard chronological split of 12/4/4 months for train/validation/test, standardizes using training-split statistics only, builds sliding windows of length $L + H$, and computes the cycle index as $t \bmod W$. No observations are excluded beyond the windowing boundaries. The CSV is downloaded from the link in the authors’ repository and placed under `paper_code/CycleNet/dataset/ETTh1.csv`. With $L=96$, the resulting window counts are $\{8449, 8353, 8209\}$ train, $\{2785, 2689, 2545\}$ val, and $\{2785, 2689, 2545\}$ test for $H \in \{96, 192, 336\}$ respectively.

¹<https://github.com/ACAT-SCUT/CycleNet>

3.3 Hyperparameters

We did not perform a search; values come from the paper, the official code, or were fixed once for the project (Table 2). Model selection on ETTh1 uses validation MSE: the best validation checkpoint is then evaluated once on the test set.

Table 2: Main hyperparameters and their source.

Hyperparameter	Value(s)	Source
Look-back L , ETTh1	96	Paper
Horizon H , ETTh1	{96, 192, 336}	Subset of paper horizons
Cycle length W , ETTh1	24 (default), sweep {12, 24, 48, 168}	Paper / our sensitivity study
Synthetic p , L , H	24, {6, 24}, 192	Our stress test
Batch size	32	Our choice
Optimizer (Linear/MLP)	Adam, lr 10^{-3}	Our choice
Optimizer (Transformer)	AdamW, lr 10^{-4}	Stabilizes attention training
Epochs	20	Our choice
Random seed	42	Reproducibility
Instance normalization	RevIN-style	Paper

3.4 Experimental setup

All models are trained with mini-batch SGD on the MSE objective; the target is the last H steps returned by the loader, $Y = \text{batch_y}[:, -\text{pred_len}:, :]$. We report MSE and MAE on the test set, averaged over windows, prediction steps, and variables. Because ETTh1 is standardized with training-split statistics, ETTh1 metrics are on the standardized scale. Each configuration is run once with seed 42 — we therefore report single-run results, not means over seeds, which is a limitation discussed in Section 5.

3.5 Computational requirements

All runs were executed locally on a MacBook with Apple Silicon M4, using PyTorch with MPS acceleration where available. Linear and MLP runs take a few minutes each; Transformer-style runs take tens of minutes. No external cluster was used. Total cost is on the order of a few hours of laptop time. Exact per-run wall-clock times were not logged systematically.

4 Results

Our results support the four claims of Section 2. On synthetic data, CycleNet-Linear matches the Linear baseline when a full cycle is visible and improves MSE by $\sim 90\%$ when only a partial cycle is visible. On ETTh1, CycleNet-Linear consistently beats the Linear baseline across horizons, with relative MSE gains growing from 2.9% to 4.0% as H grows from 96 to 336. Cycle-length sensitivity favors daily-scale W . RCF also helps the MLP and Transformer-style backbones, although the best overall model in our setup remains CycleNet-Linear.

4.1 Results reproducing the original paper

4.1.1 ETTh1 horizon study (Claims 1 and 2)

Fixing $L=96$, $W=24$, we compare Linear and CycleNet-Linear across $H \in \{96, 192, 336\}$ (Table 3). CycleNet-Linear improves over Linear at every horizon, and the relative gain grows with H .

At $H=96$ our numbers closely match the paper’s CycleNet/Linear+RevIN entry for ETTh1 (0.377 MSE, 0.391 MAE vs. our 0.3774 / 0.3916); the no-RCF baseline is also close (0.385 / 0.393 vs. our 0.3887 / 0.3956).

Table 3: ETTh1 horizon study, $L=96$, $W=24$. Lower is better.

H	Linear MSE	CycleNet MSE	Δ MSE	Linear MAE	CycleNet MAE
96	0.3887	0.3774	−2.90%	0.3956	0.3916
192	0.4403	0.4246	−3.57%	0.4249	0.4188
336	0.4822	0.4627	−4.04%	0.4467	0.4392

We do not claim a full reproduction of the paper’s benchmark table, which spans more datasets, horizons, and baselines than we cover.

4.1.2 Cycle length sensitivity (Claim 3)

Fixing $L=96$, $H=336$ and sweeping $W \in \{12, 24, 48, 168\}$ (Table 4), daily-scale cycles ($W=24$ and $W=48$) perform best, while half-day ($W=12$) and weekly ($W=168$) cycles are clearly worse. This matches the paper’s choice of $W=24$ for ETTh1 and confirms that the cycle table is not merely adding capacity: alignment with a meaningful periodicity is required.

Table 4: Cycle length sensitivity for CycleNet-Linear, $L=96$, $H=336$.

W	Interpretation	Test MSE	Test MAE
12	half-day	0.4742	0.4423
24	daily	0.4627	0.4392
48	two-day	0.4636	0.4391
168	weekly	0.4794	0.4476

4.1.3 MLP backbone (Claim 4)

We replace the Linear backbone with a two-layer MLP at $L=96$, $H=336$, $W=24$ (Table 5). CycleNet-MLP improves over MLP, but only slightly, and the simple CycleNet-Linear remains the strongest model. The paper’s finding that CycleNet-MLP is generally stronger than CycleNet-Linear is therefore not reproduced in our setting.

Table 5: MLP vs. Linear backbones on ETTh1, $L=96$, $H=336$, $W=24$.

Model	Test MSE	Test MAE
Linear	0.4826	0.4466
CycleNet-Linear	0.4616	0.4379
MLP	0.4962	0.4552
CycleNet-MLP	0.4922	0.4523

4.2 Results beyond the original paper

4.2.1 Synthetic sanity check: full vs. partial cycle (Claim 1)

To isolate the RCF mechanism we generate data with a known cycle $p=24$ and compare two settings (Table 6): an input window that contains a full cycle ($L=24$) and one that contains only a partial cycle ($L=6$), both with $H=192$ and $W=24$. When the full cycle is visible, the Linear model already learns the periodic mapping and CycleNet-Linear matches it. When the input is shorter than one cycle, the Linear baseline fails but CycleNet-Linear preserves low error, reducing MSE by $\sim 90.45\%$ and MAE by $\sim 69.49\%$.

Table 6: Synthetic periodic signal, $p=24$, $H=192$.

Setting	Model	Test MSE	Test MAE
Full cycle ($L=24$)	Linear	0.0258	0.1277
	CycleNet-Linear	0.0260	0.1279
Partial cycle ($L=6$)	Linear	0.2804	0.4256
	CycleNet-Linear	0.0268	0.1298

4.2.2 CycleiTransformer ablation (Claim 4)

The authors’ repository ships `CycleiTransformer.py` but not a matched no-cycle `iTransformer.py`, so we built the ablation by removing only the cycle-subtraction and cycle-restoration operations. At $L=96$, $H=336$, $W=24$, CycleiTransformer improves over the matched no-cycle baseline by small but consistent margins on both metrics (Table 7).

Table 7: Transformer-style backbone with and without cycle modeling.

Model	Test MSE	Test MAE
Transformer-style, no cycle	0.4807	0.4570
CycleiTransformer	0.4752	0.4523

5 Discussion

The four claims hold in our reproduction, with one nuance. Claims 1 and 2 (RCF improves over no-cycle backbones; the benefit grows with H) are clearly supported on ETTh1 and dramatically supported on the partial-cycle synthetic experiment. Claim 3 (cycle length matters) is supported by the sensitivity sweep. Claim 4 (RCF is backbone-agnostic) holds in direction but not in magnitude: CycleNet-MLP and CycleiTransformer beat their respective no-cycle baselines, but neither beats CycleNet-Linear in our setup. The cleanest takeaway is that the gains come from *explicit* periodic modeling, not from added capacity.

The reproduction has clear limits: single-seed runs instead of $\text{mean} \pm \text{std}$ over seeds, ETTh1 only rather than the full benchmark suite, and a reduced Transformer configuration to keep runs laptop-feasible. The horizon study omits $H=720$ for the same reason.

5.1 What was easy

RCF is a small, well-described mechanism, so the core reimplementaion was direct. The authors’ loader handled chronological splits, normalization, sliding windows, and cycle indexing consistently with the paper, which made the ETTh1 setup low-risk.

5.2 What was difficult

Aligning the cycle-table phase with the input and prediction windows: an off-by-phase index breaks the method silently and was the source of an early failure in the synthetic experiment. Building a fair Transformer-style no-cycle baseline was also non-trivial because the repository does not ship one. Single-seed runs and the absence of a hyperparameter search are limitations we accepted to keep the study focused.

5.3 Communication with original authors

We did not contact the original authors. Our reimplementaion relied on the paper and the public repository.

6 Conclusion

CycleNet’s central idea — learn a recurrent periodic component, subtract it, forecast the residual, add it back — held up in our reimplementation. On ETTh1 with $L=96$, $H=96$, $W=24$, our CycleNet-Linear lands within 0.001 MSE of the paper’s reported number; gains over the Linear baseline grow with horizon, and cycle-length sensitivity behaves as expected. Stronger backbones with RCF also beat their no-cycle counterparts, although the simple CycleNet-Linear remains our strongest model. Natural extensions include multi-seed evaluation, the full ETT/Electricity/Traffic benchmark suite, and combining RCF with inter-channel modeling.

Member contributions

Ferdinando Giordano led the implementation: he reimplemented the RCF mechanism, built the synthetic and ETTh1 pipelines, ran the horizon, cycle-length, MLP and CycleiTransformer experiments, and integrated the results into the report. **Gianluca Viviano** validated the experimental setup, audited the data-loading pipeline and split alignment, and helped interpret the horizon and cycle-length results. **Alan Copa** contributed the background and related-work framing, organized the report structure, and prepared tables and the discussion of limitations.

References

Shengsheng Lin, Weiwei Lin, Xinyi Hu, Wentai Wu, Ruichao Mo, and Haocheng Zhong. Cyclenet: Enhancing time series forecasting through modeling periodic patterns. In *Advances in Neural Information Processing Systems*, 2024.